Reinforcement Learning – Final Project

MiniGrid-Dynamic-Obstacles Navigation with Deep Reinforcement Learning

Course: Reinforcement Learning

Submitted to: Prof. Moshe Butman
Institution: Reichman University
Submitted by:
Igor Nazarenko - 322029158,
Yuval Amit - 203839907

Submission Date: July 28, 2025

Table of Contents

Main Report Section

1. **Introduction and Motivation**
2. **Environment Analysis and Problem Setup**
3. **Methodology / Modeling Approach**
4. **Experiments, Ablation & Results**
5. **Discussion and Takeaway's**

Figures and Tables Section

- **Figure 3.1:** Utility Wrapper – SimpleResizeObs
- **Figure 3.2:** Utility Wrapper – SimpleFrameStack
- **Figure 3.3:** make_env function code snippet with all wrappers and preprocessing steps
- **Figure 3.4:** Comparative training strategy breakdown: A2C vs. Dueling DRQN
- **Figure 3.5:** Replay and sequence sampling workflow diagram for prioritized recurrent buffer
- **Figure 3.6**: n-step TD target computation logic and sequence alignment
- **Figure 3.7:** Reward shaping and exploration strategy flowchart
- **Figure 4.1:** Final evaluation summary (success rate, return, episode lengths, outcomes)
- **Figure 4.2:** Training progress – smoothed success rate and episode length curves for A2C and DRQN
- **Figure 4.3:** DRQN diagnostics – TD loss curve and Q-value distribution
- **Figure 4.4:** DRQN replay buffer statistics (sampling priority histogram and distribution metrics)
- **Figure 4.5:** Replay buffer ablation results – uniform vs. prioritized replay (performance comparison)
- **Figure 4.6:** Reward shaping and exploration bonuses – ablation impact on success and learning curve

Appendices Section

- **Appendix A: Dueling DRQN (Final Agent)**
    - **A.1:** Architecture Summary – convolutional encoder, GRU core, dueling Q-head
    - **A.2:** Training Pipeline – DRQN training stage descriptions
    - **A.3:** Hyperparameters – replay, optimization, GRU, and ε-decay configuration
- **Appendix B: A2C (Baseline Agent)**
    - **B.1:** Architecture Summary – shared CNN encoder with actor-critic heads
    - **B.2:** Curriculum learning architecture table
    - **B.3:** Hyperparameters – learning rate, entropy, value loss weight, parallel envs
    -
- **Appendix C: Evaluation Setup**

1. Introduction & Motivation:

This project explores the challenge of learning effective policies in high-dimensional, partially observable environments with sparse rewards and dynamic elements. Our testbed, **MiniGrid-Dynamic-Obstacles-16x16-v0**, presents a complex navigation task due to its large state space, moving obstacles, and minimal feedback.
These conditions closely reflect real-world problems in sequential decision-making under uncertainty, where agents must learn from limited sensory input, rely on memory, and plan over long horizons. The study focuses on evaluating reinforcement learning algorithms, specifically, memory-based value learning and policy gradient methods, under these constraints, aiming to understand their sample efficiency, robustness, and ability to generalize across varied initializations.

2. Environment and Problem Setup:

We used the **MiniGrid-Dynamic-Obstacles-16×16-v0** environment, a partially observable maze where the agent must reach a fixed goal while avoiding moving obstacles. The task is framed as a **Partially Observable Markov Decision Process** (POMDP) with several RL-specific challenges:

- **Partial Observability**: The agent only sees a limited 7×7 RGB window in the direction it faces.
- **Stochastic Layouts**: Each episode begins with a randomly generated map and random obstacle motion.
- **Sparse Rewards:** A reward of +1 is given only upon reaching the goal; all other transitions yield zero.
- **Hard Termination:** Episodes end on success, collision, or timeout.

The agent operates in a **discrete action space**: {turn left, turn right, move forward}, with visual input provided by the RGBImgPartialObsWrapper, simulating first-person vision.
Together, these elements require **temporal reasoning** and **robust exploration**. The 16×16 map size and dynamic hazards extend the planning horizon, making the task significantly harder than smaller or static MiniGrid variants.

3. Methodology / Modeling Approach

To address the challenges of the Dynamic-Obstacles environment, we implemented and evaluated two learning-based agents: an Advantage Actor-Critic (A2C) and a Dueling Deep Recurrent Q-Network (Dueling DRQN). Each model was trained on image-based partial observations and incorporates different strategies to handle partial observability, sparse rewards, and dynamic hazards. This section outlines the preprocessing steps, model architectures, exploration schemes, and training techniques used in our experiments.

a. Preprocessing:

Both agents received pixel-based inputs using MiniGrid's RGBImgPartialObsWrapper, producing 7×7×3 RGB images. We used ImgObsWrapper to extract the RGB tensor and applied a custom SimpleResizeObs wrapper to resize images to 64×64×3 using nearest-neighbor interpolation, standardizing spatial input and improving CNN receptive fields.
To encode temporal information, we employed different approaches: A2C used SimpleFrameStack (n_stack=4) to provide short-term memory via stacked frames, while DRQN omitted stacking (n_stack=1) and relied on its recurrent GRU module for temporal reasoning. DQN-based agents used earlier in our experiments also utilized frame stacking. The complete wrapper pipeline is shown in Figures 3.1–3.3 (SimpleResizeObs, SimpleFrameStack, make_env).

b. Model Architectures, Training Design & Strategies (Figure 3.4 for strategy comparison; Appendixes A, B for architectures, training configuration and hyperparameters):

We implemented two agents, A2C and Dueling DRQN, both trained to solve partially observable MiniGrid tasks from raw pixel inputs. While A2C was designed as a lightweight on-policy baseline, it ultimately failed to learn an effective policy. In contrast, our final DRQN agent incorporated advanced techniques to improve temporal reasoning, sample efficiency, and reward shaping, achieving significantly better results.

A2C (Baseline Agent) (Figure 3.4 for direct comparison with DRQN training strategies)

- **Architecture** (Appendix B.1 for architecture table):
  - The initial A2C model used stacked frames to encode temporal context without recurrence, which proved insufficient in the partially observable setting. To address this, we introduced a multi-layer GRU

after the CNN encoder, enabling the policy to track hidden state information and temporal dependencies. While this extension improved pattern recognition, the agent still failed to learn a stable policy, likely due to A2C's on-policy nature, limited sample reuse, and the challenges of sparse rewards.

- **Training**: (Appendix B.3 for hyperparameters table):
  - The A2C agent was trained on 16 parallel vectorized environments using the Advantage Actor-Critic framework enhanced by Generalized Advantage Estimation (GAE-λ). Observations consisted of stacked image frames processed by a GRU-based network to compute action logits, value estimates, and updated memory states. Actions followed an ε-greedy strategy with linearly decaying exploration, balancing exploration and exploitation over time. Shaped rewards were used to compute normalized advantages. The value loss included PPO-style clipping, and entropy regularization with cosine decay promoted policy diversity. A KL-divergence monitor adjusted the learning rate and triggered rollback mechanisms to maintain stability.
- **Exploration**:
  - Entropy regularization discouraged early policy collapse, while a single-phase ε-greedy schedule linearly decayed exploration from high to low.
- **Reward Signal**:
  - The reward combined the sparse success signal with shaping components: a small step penalty (to encourage progress), distance-based shaping (to reward approach), and a revisit penalty (to discourage loops).
- **Curriculum Learning Strategy (**Appendix B.2 for Curriculum learning architecture table):
  We use multi-phases approach where the model first train on simple 5x5 empty minigrid and then continue to more complex environments up until 16x16 dynamic obstacles. All phases trained sequentially using the same vectorized A2C model with 16 parallel environments, incrementally scaling complexity and reusing learned parameters.

<u>Dueling DRQN (Final Agent)</u>

- **Architecture** (Appendix A.1 for architecture table)
  - CNN encoder followed by a **GRU** backbone to capture sequential context.
  - A **dueling head** decomposes Q-values into value and advantage, stabilizing learning under partial observability.
- **Warm Start**:
  - Enabled model reuse by loading compatible weights from pretrained agents (e.g., trained on smaller grids). This accelerated convergence and stabilized early training.
- **Exploration** (Figure 3.7):
  - **Two-phase ε-greedy** policy with gradual decay from full exploration to fine exploitation.
  - **Count-based bonus** encouraged novel state discovery early in training.
  - **First-visit rewards** amplified early state coverage.
- **Reward Shaping**(Figure 3.7):
  - **Potential-based shaping** using negative Manhattan distance to goal.
  - **Distance-scaled step penalties** imposed mild time pressure, gradually relaxed as training progressed.
  - **Collision penalties** scaled down rewards when bumping into walls or objects.
- **Training Pipeline**(Figure 3.6 for n-step logic; Appendix A.2-A.3 for full pipeline):
  - Optimized off-policy using **Huber loss** on **n-step TD targets** (with flexible n) (Figure 3.6).
  - **Polyak averaging** was applied to softly synchronize target and online networks.
  - **Gradient clipping** stabilized updates during training.
- **Replay & Sequence Logic**(see Figure 3.5 for sampling design and Figure 4.5 for ablation):
  - Used a **prioritized recurrent replay buffer** with fixed-length sequences and burn-in for hidden state warm-up.

- **Uniform replay** led to faster early gains, while prioritized replay improved late-stage sample efficiency via TD-error-based sampling and importance weights.
- Final Q-updates followed **Double DQN** logic, selecting actions via the online network and evaluating them with the target.

Training performance metrics and diagnostic plots (learning curves, TD loss, buffer statistics) are discussed in detail in Section 4.

4. Experiments, Ablation & Results:

To systematically evaluate learning under partial observability, we first attempted to train a lightweight A2C baseline before progressing to a recurrent DRQN enhanced for memory, exploration, and sparse reward handling. This section presents a structured sequence of experiments, ablation studies, and final evaluations of both agents. Despite extensive tuning and architectural changes, A2C consistently failed to converge. Its on-policy nature limited sample reuse and long-term credit assignment.

We experimented with frame stacking and added recurrence (first LSTM, then GRU) to improve temporal awareness, but instability remained. We also applied reward shaping and curriculum learning across increasingly complex environments (5×5 to 16×16), yet these efforts did not resolve the model's weaknesses.

a. Evaluation Protocol

All agents were evaluated on the unshaped MiniGrid reward ($r \in \{0, +1\}$) using deterministic greedy policies across 100 randomized, held-out seeds. Soft tie-breaking ($\tau = 0.02$) ensured consistent behavior. Evaluation conditions matched training (observation, reward, termination). See Appendix C and Figure 4.1 for evaluation setup and summary.

The main evaluation metrics were:

- **Success Rate**: Fraction of episodes where the goal was reached.
- **Episode Length**: Number of steps per episode until success, collision, or timeout.
- **Outcome Types**: Breakdown of success, timeout, and collision.
- **Unshaped Return**: Cumulative reward excluding shaping terms, used for fair comparison.

b. Training Progress and Learning Curves

For each agent, we tracked shaped return, episode length, and success rate throughout training. DRQN was trained off-policy for 100K updates using a prioritized recurrent replay buffer with Polyak averaging and n-step bootstrapping. Figures 4.2–4.4 show key curves:

- Learning curves of shaped and unshaped rewards
- Smoothed success rate and episode length
- Replay sampling metrics (for DRQN only)

c. Ablations and Intermediate Agents

To develop the final Dueling DRQN, we tested intermediate configurations and architectural decisions:

- **TinyDQN (5×5/8×8)**: Basic CNN+frame-stacking setup used for early diagnostics and pipeline validation.
- **DQN → DRQN:** Replacing stacked frames with a GRU significantly improved temporal modeling under occlusion.
- **Warm Start (8×8 → 16×16):** Using pretrained weights stabilized training on larger maps and improved initial performance.
- **Replay Ablation:** Prioritized replay ($\alpha = 0.6$) enhanced sample efficiency. Uniform replay yielded robust policies but slower learning (~124 vs. ~70 steps). Burn-in stabilized recurrence (Figure 4.5).
- **Shaping & Exploration Bonuses:** Controlled experiments evaluated each shaping component (potential, collision penalty, count bonus). These mitigated reward sparsity and accelerated learning (Figure 4.6).

d. Comparative Summary

Final evaluation used the same 16×16 test protocol for both agents. A2C, despite significant engineering, remained unstable and failed to generalize effectively. In contrast, the DRQN agent, equipped with recurrence, reward shaping, and prioritized replay, achieved high success rates, shorter episodes, and consistent policy behavior.

Figure 4.1 compares (best DRQN and unniform DRQN):

- Success rate distributions
- Episode duration histograms
- Outcome types
- Final unshaped returns

Our results show that Dueling DRQN clearly outperforms A2C, validating the need for memory, experience replay, and structured reward signals in high-dimensional POMDPs. These findings validate our stepwise progression and highlight the limitations of on-policy RL in sparse, dynamic tasks.

Additionally, the shaped agent reached the 80% success threshold around step 85K, approximately 30K steps earlier than the non-shaped agent, demonstrating improved sample efficiency.

5. Discussion and Takeaway's:

Our experiments indicate that combining recurrence, reward shaping, and prioritized replay is critical for learning in partially observable, sparse-reward environments like MiniGrid-Dynamic-Obstacles-16×16. This section synthesizes the core insights, highlights limitations, and distills practical lessons for future RL research.

a. Key Insights & Interpretation

- **Why DRQN Outperformed A2C**
  - DRQN's recurrent architecture provided temporal context that A2C lacked. Maintaining an internal state helped it deal with occlusion and dynamic elements. The dueling head stabilized value estimation by decoupling state value from action advantages.
- **Effectiveness of Enhancements**
  - **Reward shaping** (potential-based, collision penalties, revisit bonuses) smoothed sparse rewards and accelerated early learning.
  - **Prioritized replay** improved sample efficiency by emphasizing transitions with high TD error.
  - The **dueling structure** stabilized training under partial observability.
- **Efficiency vs. Stability Trade-offs**
  - A2C trained quickly and was simple to implement but failed to converge due to its lack of memory and sample reuse. In contrast, DRQN required more tuning and time but achieved >85% training and >95% evaluation success through better temporal modeling and stability.

b. Limitations - Despite the improvements, several challenges remain

- **Training complexity:** DRQN required careful tuning of sequence length, burn-in, and target update frequency.
- **Overfitting risk:** Prolonged training on fixed seeds may reduce generalization.
- **Reproducibility:** Sensitive to initialization and seed, requiring robust logging/checkpointing for repeatability.

c. Practical Lessons & Final Thoughts

Key takeaways for RL in partially observable environments:

- **Recurrence** is critical when memory or occlusion is involved.
- **Shaped rewards and count-based bonuses** mitigate sparse reward issues.
- **Prioritized recurrent replay** boosts convergence but adds system complexity.
- Lightweight **on-policy methods** like A2C may suffice in simple tasks but underperform in dynamic, high-dimensional POMDPs.
- **Architectural choices** (dueling heads, n-step returns) must match the task's temporal and structural demands.

These lessons underscore the need for memory-aware agents, structured exploration, and stable replay systems in scaling RL to real-world domains.

Figures and Tables:

Figure 3.1: Utility Wrapper - SimpleResizeObs:

```python
class SimpleResizeObs(gym.ObservationWrapper):
    """Resize ndarray observation (H,W,3) to target shape with nearest-neighbour.
        •   Bigger receptive field - A 56×56 crop collapses after two 3×3 conv layers; 84×84 lets a small CNN capture spatial relations.
        •   Video compatibility - 64, 80, 96, … are multiples of 16, so MP4 encoders don't warn.
        •   Consistent input - Every experiment, seed, and grid size produces the same tensor size for the network."""
    def __init__(self, env, shape=(84, 84)):
        super().__init__(env)
        self.shape = shape                # (H, W)
        h, w = shape
        # Detect whether we're dealing with a Dict or a Box
        if isinstance(env.observation_space, gym.spaces.Dict):
            c = env.observation_space["image"].shape[-1]
        else:                                       # already a Box
            c = env.observation_space.shape[-1]

        self.observation_space = gym.spaces.Box(
            low=0, high=255, shape=(h, w, c), dtype=np.uint8
        )

    def observation(self, obs: np.ndarray) -> np.ndarray:
        # Extract the RGB image
        if isinstance(obs, dict):
            img = obs["image"]
        else:
            img = obs                       # already a plain array

        # Resize (using cv2, PIL, or torch-vision - pick one)
        img = cv2.resize(img, self.shape, interpolation=cv2.INTER_AREA)
        return img
```

SimpleResizeObs is a custom Gym wrapper that resizes MiniGrid's partial RGB observations to a fixed 84×84×3 shape, standardizing inputs for CNN-based agents and enabling consistent spatial representation across environments.

Figure 3.2: Utility Wrapper - SimpleFrameStack:

```python
class SimpleFrameStack(gym.Wrapper):
    """Stack last *k* images along axis 0  →  (k,H,W,3) uint8.
        •   Partial observability - the agent cannot see behind itself; stacked frames give a short-term memory of what just left view.
        •   Motion cues - obstacle velocity (and the agent's own motion) becomes visible via frame differences, letting a plain CNN learn without an LSTM.
        •   Keeps network simple - avoids writing a custom recurrent head until we benchmark the basic CNN."""
    def __init__(self, env, k: int = 4):
        super().__init__(env)
        self.k        = k
        self.frames   = deque([], maxlen=k)
        h, w, c       = env.observation_space.shape
        self.observation_space = gym.spaces.Box(
            low = 0, high = 255, shape=(k, h, w, c), dtype=np.uint8
        )

    # ------------------------------------------------------------------ API
    def reset(self, **kw):
        obs, info = self.env.reset(**kw)
        self.frames.extend([obs] * self.k)          # fill deque
        return self._get_obs(), info

    def step(self, action):
        obs, r, term, trunc, info = self.env.step(action)
        self.frames.append(obs)
        return self._get_obs(), r, term, trunc, info
    # ------------------------------------------------------------------
    def _get_obs(self):
        return np.stack(self.frames, axis=0)
```

SimpleFrameStack stacks the last k image observations along a new axis (→ shape: k×84×84×3), providing short-term visual memory and motion cues to non-recurrent agents like DQN. This helped our early CNN-based models compensate for partial observability before transitioning to DRQN with recurrence.

Figure 3.3 make_env function code:

make_env constructs the full MiniGrid environment pipeline used in all agents. It sets the map size, seed, wraps the RGB observation to plain ndarray, optionally resizes frames (to 84×84 or 64×64), and stacks k frames if required. DRQN agents used n_stack=1 and resized to 64×64, while earlier DQN models stacked 4 frames at 84×84.

```python
def make_env(size: str            = "16x16",
             seed: int            = 0,
             resize_to: Optional[Tuple[int, int]] = (84, 84),
             n_stack: int         = 4):
    """
    size      : '5x5', '6x6', '8x8', or '16x16'
    resize_to : None  → keep raw 56×56
                (H,W) → upscale with nearest-neighbour
    n_stack   : 0 or 1 → no frame stack (DRQN)
                k>1    → stack last k frames (DQN)
    """
    env_id = f"MiniGrid-Dynamic-Obstacles-{size}-v0"
    env = gym.make(env_id, render_mode="rgb_array", highlight=False)

    # partial RGB crop → plain ndarray (H,W,3)
    env = RGBImgPartialObsWrapper(env)   # crop of what agent sees
    env = ImgObsWrapper(env)             # dict→ndarray (56,56,3)

    # optional up-/down-scaling
    if resize_to is not None:
        env = SimpleResizeObs(env, resize_to)      # e.g. (84,84,3)

    # only stack when you really want more than one frame
    if n_stack > 1:
        env = SimpleFrameStack(env, k=n_stack)     # e.g. (4,84,84,3)

    env.reset(seed=seed)
    return env
```

Figure 3.4 Comparison: A2C vs. Dueling DRQN Training Strategies:

This table contrasts the key design choices between the A2C baseline and our final Dueling DRQN agent. Unlike A2C's on-policy and stateless updates, Dueling DRQN incorporates recurrence, prioritized replay, Double Q-learning, and shaped rewards, enabling more efficient learning under partial observability

Feature	A2C	Dueling DRQN
Learning Type	On-policy	Off-policy (replay buffer)
Temporal Modeling	Frame Stacking + LSTM / GRU	GRU + burn-in sequences
Replay Buffer	Rollout buffer – n_steps x m_env	Prioritized Recurrent Replay
Target Computation	n-step TD (1-step default)	Double DQN, n-step TD
Reward Signal	Raw + Shaping	Raw + shaping + bonuses
Exploration Strategy	Entropy + ε-greedy	ε-greedy + count bonus + shaping
Target Net Sync	N/A	Polyak averaging
Update Frequency	Every n_steps	Every 4 steps (off-policy)

Figure 3.5 Replay and Sequence sampling Workfolw:

These diagrams illustrate the logic implemented in our PrioritizedRecurrentReplayBuffer class (see training notebook, Section 2-B-4). The system combines R2D2-style sequence-based sampling with TD-error-based prioritization, enabling the Dueling DRQN agent to focus training on critical transitions while maintaining temporal consistency.

The left diagram shows the conceptual flow: transitions are added to the buffer, minibatches are sampled based on priority, and TD errors are used to update priorities after each training step. The right flowchart details the operational logic, including sequence-window prioritization and importance sampling (IS) weights for bias correction.

This buffer is fully integrated into the train_drqn() loop (Section 2-C-2), supports n-step TD targets, and applies Polyak averaging and reward shaping. The approach improves sample efficiency, training stability, and convergence, as validated by our experiments.

Source: Inspired by R2D2 (Kapturowski et al., 2019) and OpenAI Baselines. Custom implementation adapted to MiniGrid environments.

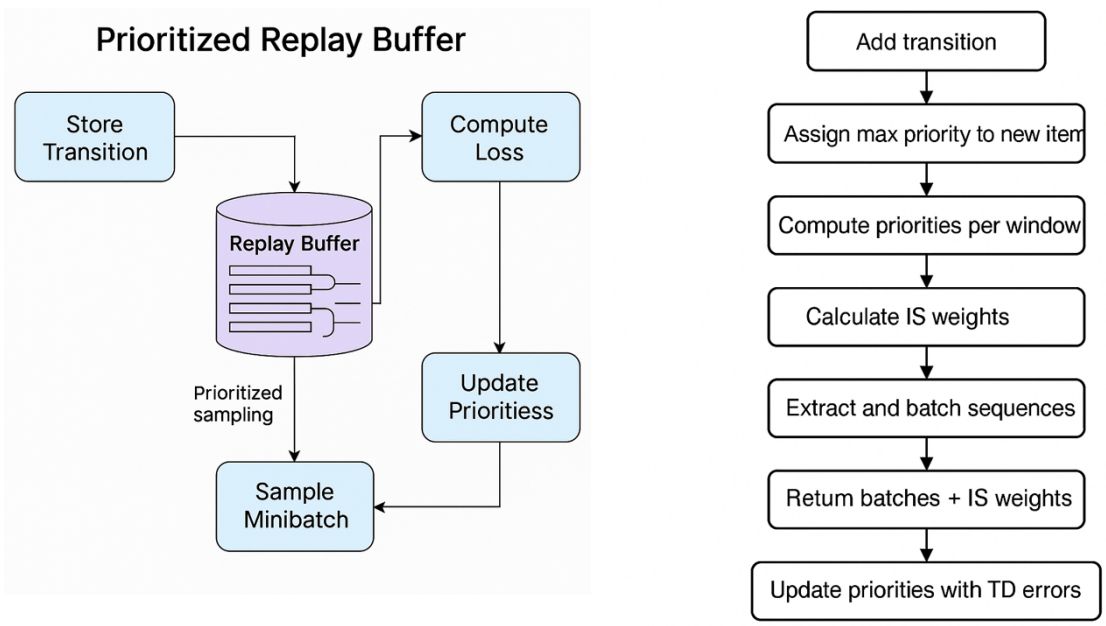


Figure 3.6 N-step TD Target Computation Flow

This diagram illustrates the computation logic for n-step Temporal Difference (TD) targets used in our Dueling DRQN training loop (see train_drqn() in Section 2-C-2). For each sampled sequence, rewards are summed over the last n steps using discounting, with terminal states zeroing future contributions. The final TD target is computed by adding the bootstrapped Q-value from the target network, discounted by γ^n. This enables stable credit assignment over extended sequences and improves convergence in partially observable MiniGrid tasks. The implementation combines n-step returns with Double DQN target selection for robust value learning.

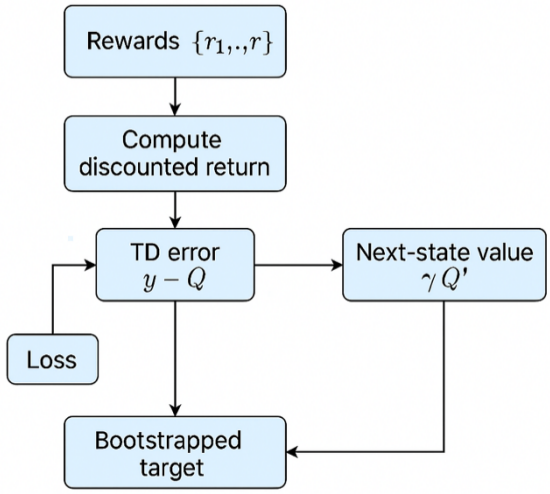


Figure 3.7 – Exploration & Reward Shaping Strategy Flow:

This table summarizes the exploration and shaping strategies embedded in our DRQN training loop. A decaying ε-greedy policy drives early exploration, complemented by a count-based first-visit bonus to incentivize novel state discovery. Reward shaping combines three elements: a potential-based shaping term using Manhattan distance to the goal, a micro-penalty scaled by progress and time, and collision penalties. Together, these mechanisms guide learning toward efficient, successful trajectories while preserving exploration incentives early in training.

Mechanism	Trigger / Condition	Effect on Reward / Behavior
ε-greedy exploration	Decays linearly from ε=1.0 → 0.05 over eps_decay steps	Promotes early exploration, stabilizes toward exploitation
Count-based bonus	Active if step < bonus_cutoff and ε > 0.10	Adds novelty bonus: \frac{β}{\sqrt{N(s) + 1}} to encourage state coverage
Potential-based shaping	Always active during training	Adds: γ\phi(s') - \phi(s), where \phi(s) = -d_\text{manhattan}(s, goal)
Distance-scaled penalty	Scaled down over time via: r_micro = α(1−f)·d/width	Penalizes long paths early in training to promote efficiency
Collision penalty	If r_env < 0 (e.g., bumping into wall or object)	Applies multiplicative penalty via collision_scale to discourage bad moves

Figure 4.1: Final Evaluation Results (Success Rate, Outcomes, Return, Length)

Evaluation summary for 100 τ-greedy episodes on MiniGrid-Dynamic-Obstacles-16×16. The prioritized DRQN agent achieved higher efficiency (fewer steps), while the uniform variant achieved perfect success at the cost of longer episodes.

Table 4.1 Evaluation summary – 100 τ-greedy episodes on 16×16

	success ±CI	avg reward	avg steps	collision	timeout
Dueling DRQN best	0.980 ±0.027	+0.902	67.4	0.020	0.000
Dueling DRQN uniform	1.000 ±0.000	+0.891	124.5	0.000	0.000

Summary table or subplots comparing(all appear in the evaluation notebook):

- DRQN (full vs without prioritized)
- Final success rate
- Outcome distribution
- Episode length histogram
- Unshaped return

Figure 4.2: Training Curves (DRQN + A2C)

These plots compare the training performance of the A2C and DRQN agents across the curriculum. The A2C agent (bottom row) shows early success in simple environments but collapses as the task complexity increases, with episode return and success rate dropping sharply and never recovering. In contrast, the DRQN agent (top row) steadily improves throughout training, achieving high success on the full 16×16 task. Episode lengths grow as the agent learns to explore, then shorten as it finds efficient paths. The shaped return signal proves critical, DRQN benefits from rich gradients, while A2C fails under sparse rewards and longer horizons.

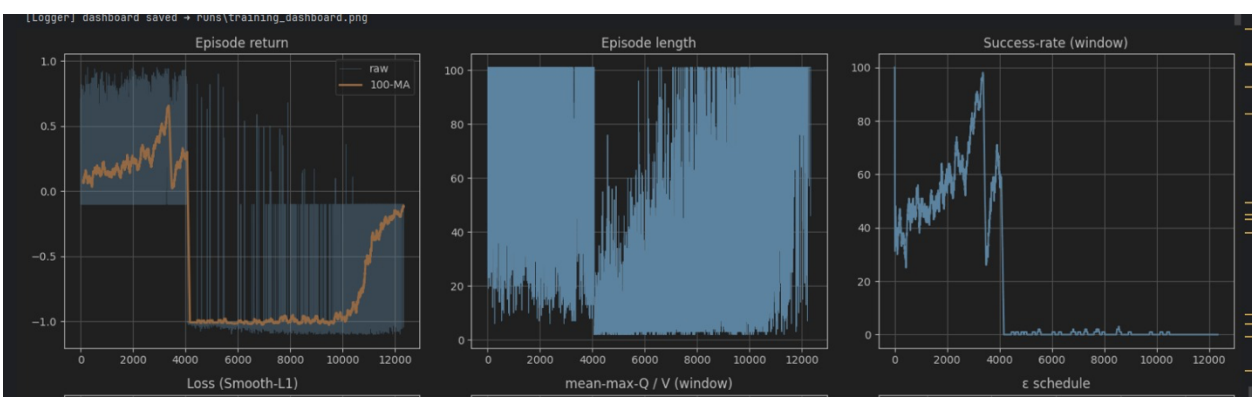


Figure 4.3 – DRQN Diagnostics: TD Loss + Q-values

Provides learning diagnostics, the TD-loss curve confirms convergence and stability, while Q-value distribution suggests bounded, well-calibrated value estimates.

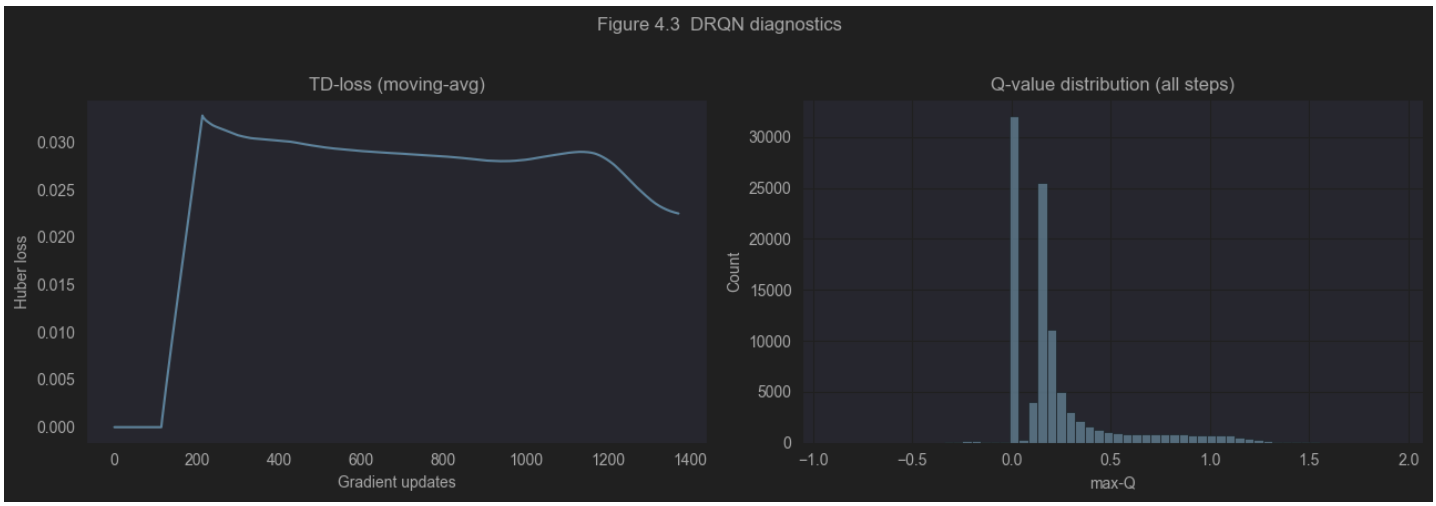


Figure 4.4 – Replay Buffer Statistics

Left: Histogram of sampled priorities indicating diversity in TD-error magnitudes used for prioritized replay.
Right: Mean priority and importance sampling weights (mean ± std) over training steps. These metrics demonstrate the evolution of prioritized sampling bias and the corresponding corrections applied during training.

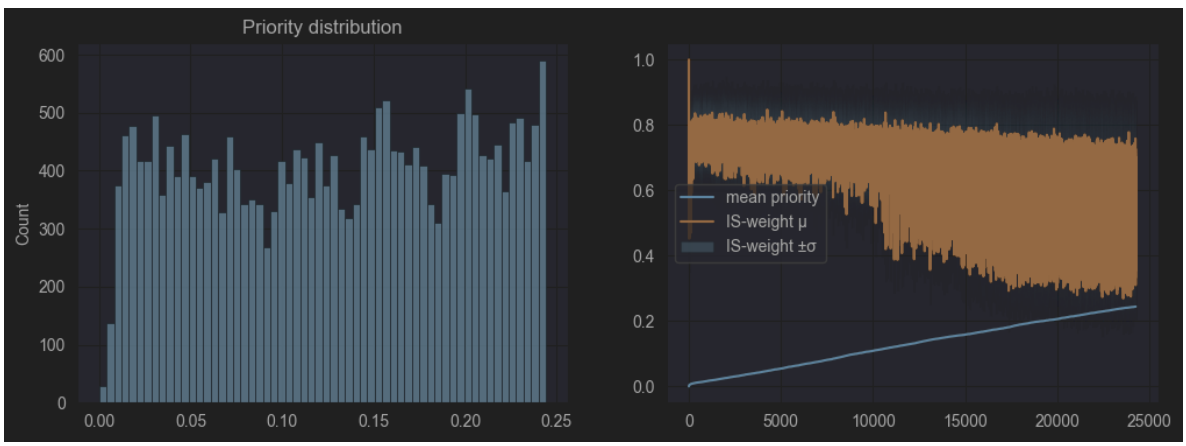


Figure 4.5 – Replay Ablation

Success-rate comparison of Dueling DRQN agents trained with Uniform vs. Prioritized replay.
Uniform buffer shows faster early convergence due to stable sequence diversity, while prioritized replay catches up later as TD-errors become more reliable. Both use the same architecture and hyperparameters.
While uniform replay eventually reached 100% success (evaluation), it did so with significantly longer trajectories (~124 steps vs. ~70 in the evaluation), highlighting an efficiency-robustness tradeoff.

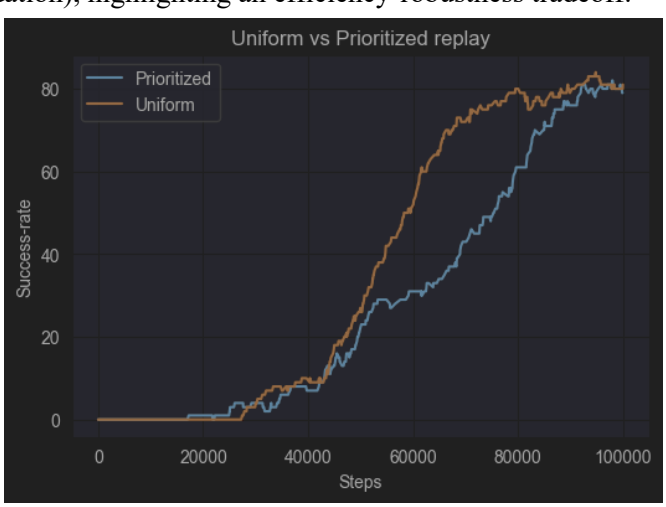
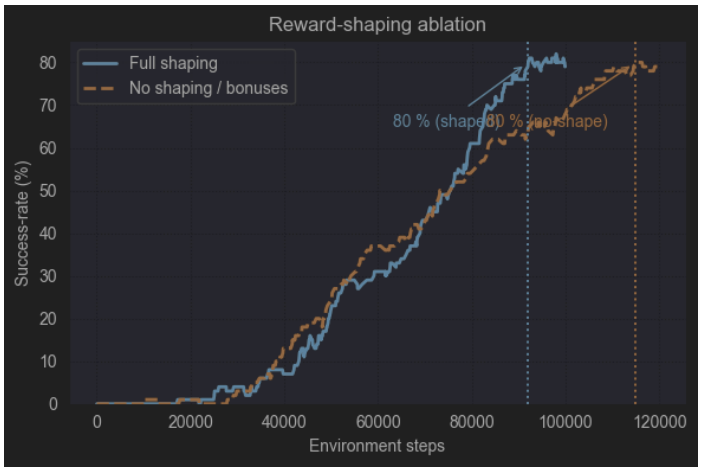


Figure 4.6 – Reward Shaping + Exploration Ablation

Shaping leads to faster convergence: the shaped agent reached 80% success around step 85K, roughly 30K steps earlier than the non-shaped agent. While final success rates were comparable (shaped: 98.0%, non-shaped: 85.0%), evaluation revealed a stark difference in policy efficiency. The shaped agent completed episodes in a mean of 67.4 steps with higher average rewards (+0.90), while the non-shaped agent averaged 586.1 steps and lower returns (+0.47). Dashed lines indicate the first point each agent surpassed the 80% success threshold.



Appendices:

Appendix A: DuelingDRQN

A.1: Architecture Summary:

Summarizes the architecture of our Dueling DRQN agent. The model uses a convolutional encoder to process raw 64×64 RGB images, followed by a GRU layer for temporal representation. The output Q-values are estimated using a dueling head that separately models the state value and action advantages (see duellingDRQN class in training notebook section 2-B-7).

Component	Configuration / Details
Input	RGB image (64×64×3), partially observable 7×7 agent view, no frame stacking
Encoder (CNN)	Conv1: 32 filters, 8×8 kernel, stride 4 Conv2: 64 filters, 4×4 kernel, stride 2 Conv3: 64 filters, 3×3 kernel, stride 1 AdaptiveAvgPool2d → Flatten
Recurrent Layer	GRU layer with hidden size 512
Dueling Head	Value Stream: Linear(512 → 1) Advantage Stream: Linear(512 → 3), for 3 discrete actions
Q-Value Output	$Q(s,a) = V(s) + (A(s,a) - \text{mean}(A(s,*)))$

A.2: DRQN Training Pipeline – Stage Descriptions

Summarizes the stages in our training loop, based on the implementation in train_drqn() (Section 2-C-2).

Stage	Description
Initialization	Setup of environment, networks, buffer, optimizer, logging tools
Action Selection	ε-greedy policy with GRU-based Q estimation
Reward Shaping	Combines Δφ potential shaping, micro step cost, novelty bonus, and collisions
Replay Buffer	Adds transitions into prioritized recurrent replay buffer
Learning Step	Sequence sampling → Q and TD target computation (n-step) → weighted loss
Priority Update	TD-error used to update buffer sampling priorities (PER)
Target Sync	Soft Polyak averaging for target network updates
Logging & Evaluation	Logs metrics, saves checkpoints, and evaluates periodically

A.3: Training Hyperparameters:

Lists all training hyperparameters used in our Dueling DRQN agent. The agent was trained off-policy using prioritized recurrent replay with Double DQN updates and reward shaping. Exploration was handled via a staged ε-greedy schedule, and performance was stabilized using Polyak averaging and gradient clipping.

Hyperparameter	Value / Setting
Optimizer	Adam
Learning Rate	5e-5
Batch Size	64
Discount Factor (γ)	0.95
Target Update Strategy	Polyak averaging ($\tau = 0.005$)
Q-Update Method	Double DQN (action from online net, value from target net)
Replay Buffer	Prioritized recurrent replay buffer (capacity = 100,000)
Prioritization α	0.6
IS Correction β	Annealed from 0.4 → 1.0 over 200,000 steps
n-step TD Return	1-step (with option to increase to 2-step in ablations)
Sequence Length	16 steps
Burn-in Length	12 steps
Gradient Clipping	Norm ≤ 10
Exploration Strategy	Two-phase ε-greedy: 1.0 → 0.1 (fast), 0.1 → 0.05 (slow decay)
Reward Shaping	Manhattan potential shaping, distance-scaled step penalty, collision penalty (×1.5)
Count-Based Bonus	+0.02 / sqrt(N+1), cutoff at 50k steps
First-Visit Reward Boost	Enabled (early phase only)
Max Episode Length	250
Total Training Steps	100,000

Appendix B: A2C

B.1: Architecture Summary:

A2CNet consists of:

1. **CNN torso**: 3 convolutional layers with 32→64→64 filters and kernel/stride settings of (8×8/4), (4×4/2), and (3×3/1), followed by adaptive average pooling to 4×4 and flattening.
2. **Fully connected layer**: a linear projection to a 256-dimensional embedding.
3. **Recurrent core**: a 2-layer GRU with hidden size 256, operating on batch-first input.
4. **Output heads**:
   - **Policy head**: linear layer mapping from 256 to 7 for action logits.
   - **Value head**: linear layer mapping from 256 to scalar value estimates.

Component	Configuration / Details
Input	RGB image (84×84×3) or stacked frames [k×84×84×3]
Encoder (CNN)	Conv1: 32 filters, 8×8 kernel, stride 4 Conv2: 64 filters, 4×4 kernel, stride 2 Conv3: 64 filters, 3×3 kernel, stride 1 AdaptiveAvgPool2d (4×4) → Flatten
Fully Connected	Linear layer: 1024 (=64×4×4) → 256 hidden units with ReLU
Recurrent Layer	GRU with 2 layers, hidden size 256, batch-first format
Output Heads	Policy Head: Linear(256 → 7) for action logits Value Head: Linear(256 → 1)
Output	Action logits $\pi(a)$

B.2: Curriculum learning architecture table:

Stage	Size	Environment -Type	Steps	Purpose
1	5x5	Empty	300k	Basic movement in small grid
2	5x5	Dynamic Obstacles	300k	Introduced obstacle avoidance
3	8x8	Empty	800k	Extended navigation complexity
4	8x8	Dynamic Obstacles	800k	Advanced path planning with obstacles
5	16x16	Dynamic Obstacles	2M	Long-term navigation in large, complex environment

B.3: Training Hyperparameters:

Hyperparameter	Value / Setting
Optimizer	Adam
Learning Rate	1e-3 - 5e-5
N_envs	8-32
Discount Factor (γ)	0.95
Q-Update Method	On-policy
Replay Buffer	Rollout buffer – n_steps (8-64) x (8-32) m_env
n-step TD Return	1-step (with option to increase to 2-step in ablations)
Sequence Length	16 steps
Frame Stacking	4 frames
Gradient Clipping	Norm ≤ 0.5 or 1
Exploration Strategy	Two-phase ε-greedy: 1.0 → 0.1 (fast), 0.1 → 0.05 (slow decay)
Reward Shaping	Manhattan potential shaping, step penalty, Revisit penalty (×1.5)
Count-Based Bonus	+0.02 / sqrt(N+1), cutoff at 50k steps
Max Episode Length	4 x h x w
Total Training Steps	4M

<u>Appendix C: Evaluation Setup Summary</u>

Component	Details
Environment	MiniGrid-Dynamic-Obstacles-16×16-v0 (RGBImgPartialObs + ImgObs wrappers, 64×64×3 input)
Episodes	**100** evaluation episodes per agent
Episode limit	MiniGrid default for 16×16: **max 1024 steps** (episode terminates earlier on success, collision, or timeout)
Exploration (τ)	Soft τ-greedy **τ = 0.02**; deterministic arg-max with random tie-break when ΔQ < τ
Policy type	Dueling DRQN (CNN + GRU + dueling head) / A2C baseline
Action selection	τ-greedy on Q-values (DRQN) or π logits (A2C); no ε-noise during evaluation
Reward type	**Unshaped** MiniGrid reward *(r ∈ {0,+1}),* all shaping terms disabled
Random seed	New seed drawn from **U(0, 1 000 000)** each episode ⇒ disjoint from training seeds
Video recording quota	At most **2 success**, **1 collision**, **1 timeout** MP4s per run; frames show full map with agent's 7×7 view **tinted**
Evaluation function	evaluate_agent(…) in evaluation notebook
Returned metrics	returns, lengths, succ_rate, coll_rate, timeout_rate, mean_return, mean_length *(JSON-exportable)*
Outcome categories	succ (goal reached) · coll (collision) · tmo (timeout)